```
  def deactivated(total: Int): Behavior[Command] = {      ❸
    Behaviors.receive { (context, message) =>
      message match {
        case Increase =>
          context.log.info(s"wallet is deactivated. Can't increase")
          Behaviors.same
        case Deactivate =>
          Behaviors.same
        case Activate =>
          context.log.info(s"activating")
          activated(total)
      }
    }
}
```

❶ Default behavior

❷ Activated behavior

❸ Deactivated behavior

In this section, you have seen how to create an FSM in Akka Typed. The Akka primitives already have the semantics of an FSM; you could say that the axioms of the actor model contain these semantics. You saw this axiom in chapter 1: “An actor must designate the behavior to be used for the next message it receives.” Because this axiom is implemented directly in Akka, you can create an FSM with nothing but its basic building blocks—its behaviors.

In a concurrent environment, thinking in terms of time is error-prone. Akka simplifies this task. Instead of making an actor wait (as you might know from threads), you can schedule messages to it.

2.5 Scheduling a message

Let’s implement another wallet that is very similar to the previous one. Everything is the same except that with this wallet, you can